Ray Tracing Assignment 2:
Optical Laws, Anti-Aliasing and Texture Mapping

September 29, 2026

Computer Graphics

Information

This assignment consists of two parts: a theoretical part (1 point) and a practical part (9 points + 1 possible bonus point). As such it's possible to earn 11 points in total. The theoretical part should be submitted as a PDF on Brightspace.

The practical part will be submitted through Themis, available on `https://themis.housing.rug.nl`. Before submitting your code, **make sure to enroll in a group with your partner first!** You absolutely need to do so before submitting any code: old submissions will not be transferred to the group and this will clutter the view when grading. Only in exceptional cases can you change groups, but only after informing the teaching assistants beforehand.

Working with Themis

This assignment consists of multiple sub-assignments in which you will be repeatedly adding new functionality to a given framework. After completing a sub-assignment, compress your project into a `zip` or `tar.gz` file and submit it to the corresponding sub-assignment on Themis. Include **only** the `CMakeLists.txt` and `src` directory in your archive. For Themis' judgment to work properly, it is also important that you do not modify the signature of existing `public` methods.

It may take a minute for Themis to judge your submission, especially if a sub-assignment has many test cases. After a submission, you will have to wait 5 minutes before making another submission for that sub-assignment. You have unlimited attempts at each sub-assignment. The sub-assignments are intended to be completed in order, and submitting a later solution to an earlier sub-assignment may not necessarily work.

This assignment does not have code quality/documentation points. That said, try to keep your code clean of course.

1 Theoretical questions (1.0 point)

This theoretical question should be submitted with a PDF file on Brightspace. For your submissions, you should use LaTeX. For drawings/diagrams, consider using Draw IO, Inkscape or TikZ. However, you can also include hand-drawn diagrams (provided that they are legible). Please write your answers as a fraction, rather than a decimal number (e.g. write $\frac{2}{7}$ instead of 0.285...). Make sure to explain your answers and show your mathematical derivations.

1. **(0.5 points)** Consider the following 2D scene description; see Figure 1.

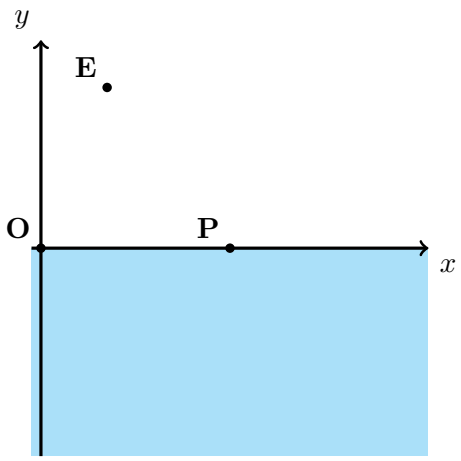


Figure 1: *The scene used in the question on refraction.*

Assume that the x-axis is the boundary between two isotropic media, where the medium above the x-axis (white) has refractive index $n_i = 1\frac{2}{5}$ and the medium below the x-axis (blue) has refractive index $n_t = 1\frac{4}{5}$. The origin is at $\mathbf{O} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, the camera is at $\mathbf{E} = \begin{pmatrix} 4 \\ 8 \end{pmatrix}$ and the point where the view ray is refracted by the blue medium is at $\mathbf{P} = \begin{pmatrix} 10 \\ 0 \end{pmatrix}$. Use Snell's law to find the **unit** vector in the direction of the refracted ray.

2. **(0.5 points)** Consider the following 2D scene description; see Figure 2.

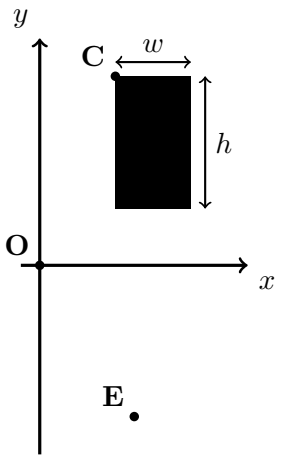


Figure 2: *The scene used in the question on reflection.*

The origin is at $\mathbf{O} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, the camera is at $\mathbf{E} = \begin{pmatrix} 5 \\ -8 \end{pmatrix}$ and the only geometric object in the scene is a rectangle with width $w = 4$ and height $h = 7$, its top-left corner is located at $\mathbf{C} = \begin{pmatrix} 4 \\ 10 \end{pmatrix}$. Assume that the camera is looking at the y-axis, which is a perfect mirror. Give the range $[m, n]$ on the y-axis where you can see the rectangle in the mirror.

2 Optical laws

In this assignment you will implement a global lighting model by extending the provided framework with support for shadows, reflection and refraction.

2.1 Shadows (2 points)

The new framework is able to read a new (optional) parameter from a scene file: `Shadows`. This is a boolean value which it stores in the `Scene` class as `renderShadows`.

Modify the lighting calculation in the `Scene::trace` function to render shadows, depending on whether `renderShadows` is `true` or `false`.

The general approach for detecting shadows is to test whether a ray from the hit point to the light source intersects other objects. Only when this is *not* the case, the light source contributes to the lighting.
Use the provided `Scene::castRay` function to cast a shadow ray. The two elements of the returned pair can be accessed using `.first` and `.second`. The first element is either a smart pointer to the hit object, or `nullptr` in case no intersection was found.

To avoid shadow acne, use the method described in the *Illumination and Shading* lecture slides. In this implementation, use the `epsilon` value provided in `scene.h`, i.e. 10^{-3}. Make sure to take the direction of the normal into account.

Figure 3 shows a sphere casting a shadow on a quadrilateral.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

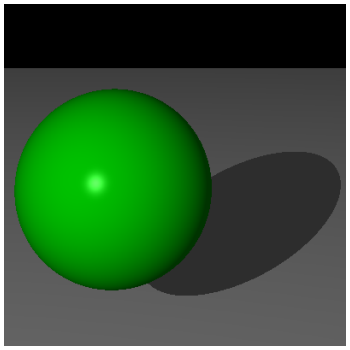


Figure 3: *The shadow of a sphere on a quadrilateral*

2.2 Specular reflection (2 points)

In the Phong illumination model, the specular component models the shininess of the object. On the surface of the object where specular highlights can be seen, the surface acts in a mirror-like fashion. This behavior can be modeled by adding support for specular reflection.

In our ray tracer, an object's material is either opaque (default) or transparent, and these two cases are handled differently. In this section, we will add specular reflections to opaque objects. Transparent objects are covered in Section 2.3.

To implement specular reflection, compute the color of the reflection as follows and add it to the `Color` as already computed according to the Phong illumination model.

1. Note how the `Scene::trace` function in the framework already distinguishes between transparent and opaque objects. Make sure the functionality to be added only applies to the latter.

2. After hitting the object, the ray should reflect off the object as if it where a (perfect) mirror. Compute the direction in which the ray is reflected.

3. Recursively trace a new ray in this direction. To avoid finding an intersection with the current object due to floating point inaccuracies, use the same method and epsilon (given in `scene.h`) used to avoid shadow acne.
   A potential problem is that the recursive ray tracing call may never terminate, for example in the case where the ray's origin is inside of a sphere. To resolve this, the `Scene::trace` function has been extended with a second parameter for the recursion depth. When calling `Scene::trace` recursively, the depth passed to it should be the current depth minus 1. If the recursion depth is defined in the scene file, it is read from there, otherwise it defaults to 0.

4. Multiply color returned by the call in the previous step by the specular coefficient of the current object. This models that the hit object is not a perfect mirror: some of the light is absorbed by the surface.

Figure 4 shows an example of specular reflections.

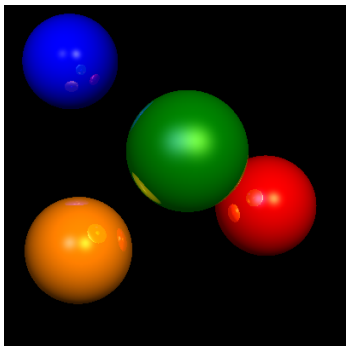


Figure 4: *Specular reflections on spheres*

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

2.3 Transparent objects (2.0 points)

In addition to having a color of their own, transparent objects reflect and refract light.

In the framework, an object is interpreted to be transparent when its material in the scene file contains the attribute `nt`: the refractive index n_t for this object. If this attribute is present in the scene file, it is stored in the `nt` data member of the `Material` class and the boolean data member `isTransparent` is set to `true`. The framework already distinguishes between transparent and opaque objects.

Implement reflection and refraction for transparent objects as described by the *Illumination and Shading* lecture slides (make sure you have the latest version). Use Schlick's approximation to compute the reflectivity, and thus the ratio between reflected and refracted light. You may assume that

- only closed objects can be transparent,
- closed objects return normals that point outwards,
- the refractive index n on the outside of the objects is 1.0,
- the objects in the scene do not intersect.

To avoid finding an intersection with the current object when casting a secondary ray, use the same method and epsilon (given in `scene.h`) used to avoid shadow acne. Also ensure that the normal points in the correct direction for this.

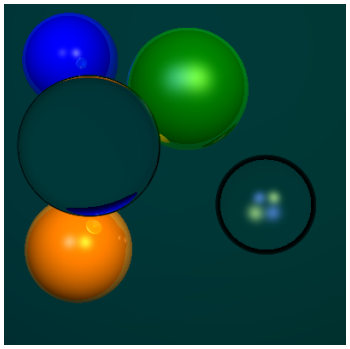


Figure 5: *Opaque and transparent spheres*

Figure 5 shows an example of transparent objects reflecting and refracting light.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

3 Anti-aliasing

Ray tracing can be regarded as sampling a geometrically described scene. As with all sampling, aliasing effects can occur, which manifest in the image as jagged edges. There are a number of anti-aliasing techniques to mitigate these effects, one of them being supersampling.

3.1 Supersampling (1.5 points)

Supersampling is a method of anti-aliasing in which the image is rendered at a higher resolution and then downsampled.
Modify the `Scene::render` function to support supersampling by casting multiple rays for each pixel and averaging the resulting colors. Note that the rays should go through the center of each (sub)pixel. This is shown in Figure 6 for 1×1 supersampling on the left, which is identical to regular sampling, and 2×2 supersampling on the right.

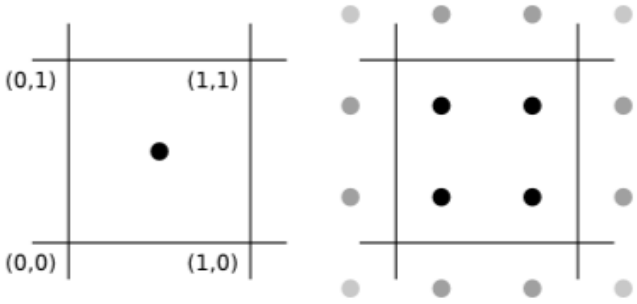


Figure 6: *Supersampling*

The supersampling factor is read from the scene file (the `SuperSamplingFactor` attribute) and stored in the data member `supersamplingFactor` in `Scene.h`. When the supersampling factor is not defined in the scene file, its value defaults to 1.
An example of 4×4 super-sampling is shown in Figure 7 (`scene01-ss.json`).

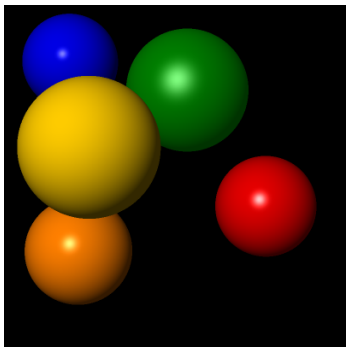


Figure 7: *An example of supersampling*

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

4 Texture mapping

This section covers texture mapping for spheres.

4.1 Fixed texture (1.5 points)

Observe how the `Material` class has been extended with the new data members `bool hasTexture` and `Image texture`. Both variables are read from the scene file, if present. The `Sphere` class has been extended with an axis and an angle. These are only relevant for Section 4.2.

Implement texture mapping for spheres as follows.

1. Complete the `Sphere::toUV` function by mapping the x,y,z-coordinates of the hit to u,v-coordinates for use with a texture. Use the mapping as defined in the *Textures* lecture slides. Do **not** use the definition in the text book, as it produces different results.

2. Modify the `Scene::trace` function. If the hit object has a texture, then use the `Object`'s function `toUV` to obtain the u,v-coordinates corresponding to the hit and obtain the color from the the `Object`'s texture as such:
   `material.texture.colorAt(u, 1.0 - v)` where $u, v \in [0, 1]$.

   Note: In a PNG file, the origin $(0, 0)$ is the top-left corner, whereas the origin for the obtained texture coordinates is the bottom-left corner. This explains why the v-coordinate has to be inverted.
   If the object does not have a texture, then use the material color as before.

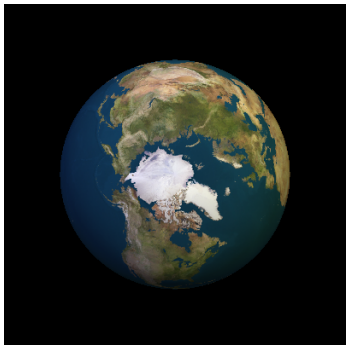


Figure 8: *The Earth texture, mapped to a sphere without rotation*

For testing your texture mapping code you may want to use `bluegrid.png` instead of a larger and more complex image.

Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

4.2 (Bonus) Rotated texture (1 point)

The `Sphere` class has been extended to store a rotation as defined by an axis and an angle. Both are read from the scene file, if these attributes are defined there.
In `sphere.cpp`, implement rotation for this shape. The sphere should rotate `angle` *degrees* counterclockwise around the axis given by `axis` (imagine you are looking at a clock along `axis` pointing to the eye/camera). The axis is given as a (possibly not normalized) vector.

Hint: Consider using quaternions.

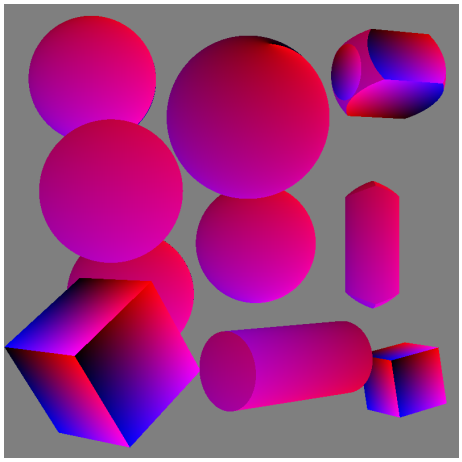


Figure 9: *The Earth texture, mapped to a rotated sphere*

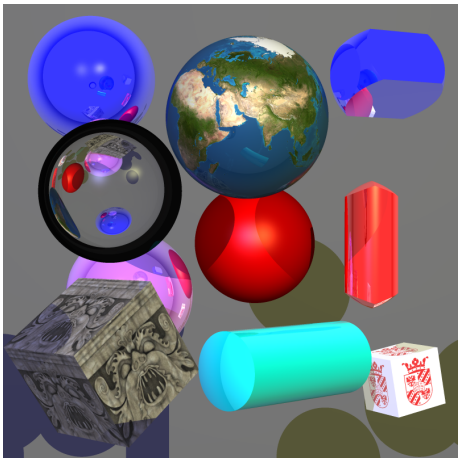
Themis: Once you have completed this part, create an archive with the `CMakeLists.txt` and `src` directory, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

5 Ideas for the competition

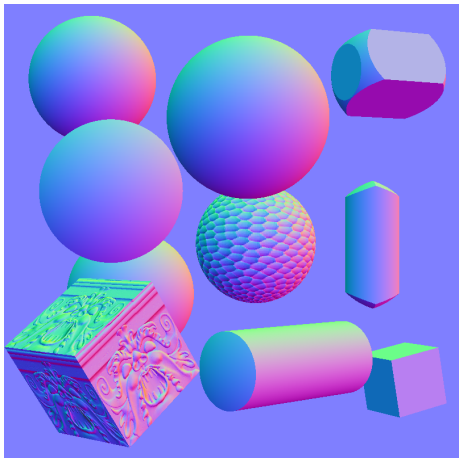
- You'll notice that the specular reflections from the scene are not blurred, like the light sources. One way to do something about this is to sample along multiple rays (around the reflection vector) and average the results. Note that for a correct result you should be careful about selecting your vectors and/or the way you average them. Hint: if you take a normal average you should select more rays in those areas where the specular coefficient is high.
- Implement normal/bump-mapping. Various examples of this technique are shown in Figure 10.



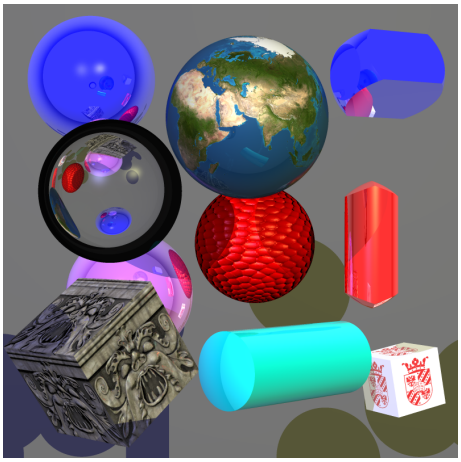
Texture coordinate buffer



Textured objects



Normal buffer



Textured objects

Figure 10: *Normal/bump maps*